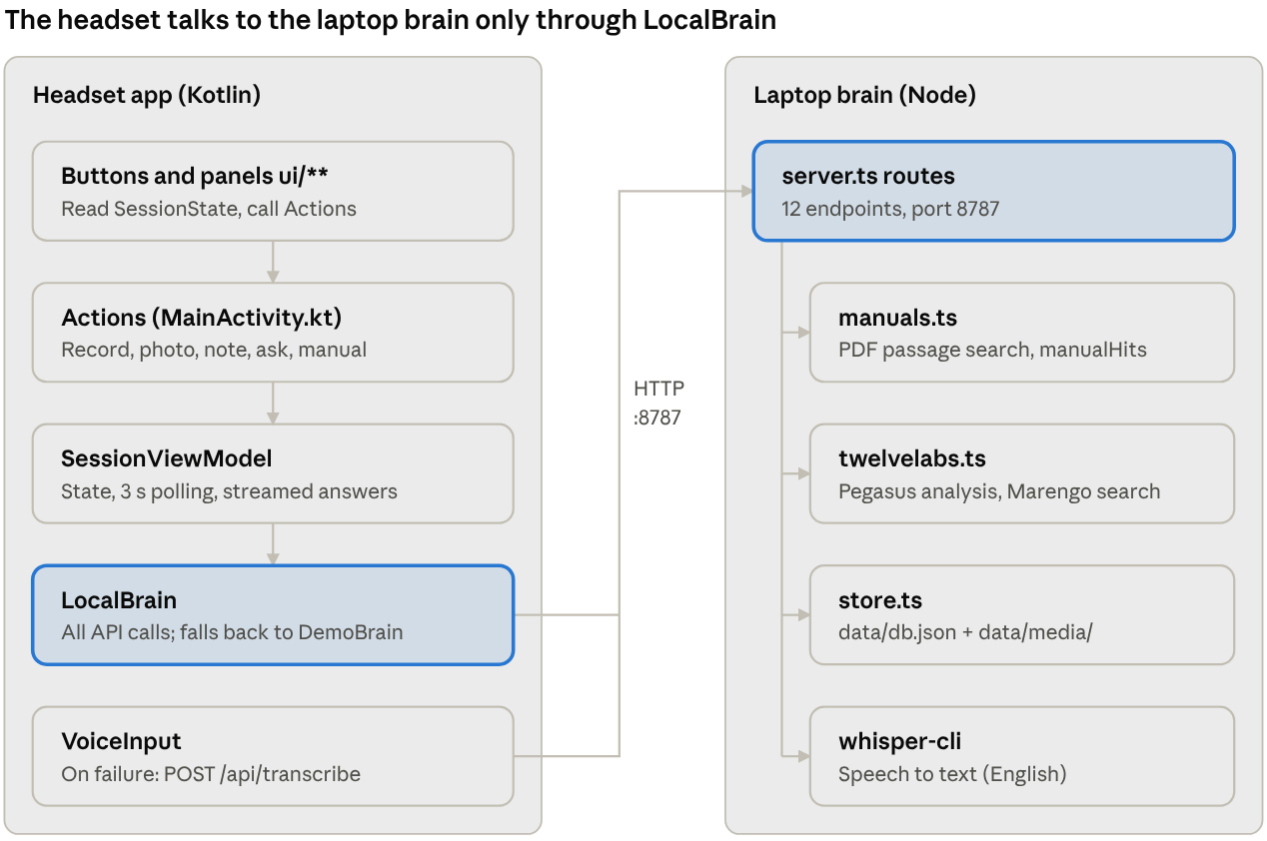
TundraXR Button & Brain API Manual

Oct 17, 2026 · @Lulu

1. Overview

The headset app talks to the Tundra brain on the laptop over HTTP (default `http://localhost:8787`). Every request goes through `LocalBrain.kt`; only the voice fallback in `VoiceInput.kt` calls the brain on its own.

- **Connection:** over USB, the brain runs `adb reverse tcp:8787 tcp:8787` every 4 seconds, so localhost on the headset is the laptop. Over Wi-Fi, build with `./gradlew installDebug -Ptundra.serverUrl=http://<laptop IP>:8787`.
- **Identity:** every headset request sends `user-agent: TundraXR/1.0`. The brain records `headsetSeenAt`, and the dashboard shows “headset connected”.
- **Offline fallback:** if the device list or a question fails, the app uses the built-in `DemoBrain` sample data. A failed upload is marked “Not sent · laptop offline” and is not retried.
- **Layers:** button (`ui/**`) → `Actions` (`MainActivity.kt`) → `SessionViewModel` → `TundraBrain` interface (`LocalBrain` / `DemoBrain`) → HTTP → brain (`server/src/server.ts`).



Architecture · 5 headset layers, 5 brain modules

Buttons never call the network directly: requests pass down to LocalBrain, then over HTTP to server.ts. The dashboard in the laptop browser talks to the same server.ts.

Timeouts

Request	Connect	Read
`GET` requests (device list, status polling)	3 s	5 s
`POST /api/captures` (media upload)	3 s	120 s
`POST /api/ask`	3 s	30 s
`POST /api/transcribe`	3 s	60 s

2. Button → API map

Only six kinds of action touch the network: recording, photos, voice notes, questions (typed, spoken or suggested), opening the manual, and loading at startup. All other buttons change local UI state only.

2.1 Buttons that call the brain

Button (where)	Action → ViewModel	API called	Input sent	Output received	What the UI does
Record / Stop (control bar)	`record` → `onRecordingStarted()`; again → `onRecordingStopped(file)`	After stop: `POST /api/captures`	MP4 file (FHD, with audio) + meta `kind:"video"`, `durationSec`	New Capture (`id`, `tl`)	Adds a video to the timeline: Sending → Analyzing → Analyzed
Take photo camera icon (control bar)	`photo` → `onShutter()` + `onPhotoTaken(file)`	`POST /api/captures`	JPEG file + meta `kind:"photo"`	New Capture	Flash effect; adds a photo to the timeline
Voice note mic (control bar)	`note` → `voice.toggle(Note)` → `onFinalTranscript()`	If recognition fails: `POST /api/transcribe`; then `POST /api/captures`	Transcribe: WAV; upload: empty body + meta `kind:"note"`, `text`	`{text}`; then new Capture	Shows live transcript; adds a note when done
Send ↑ (Ask input, or keyboard Enter)	`send` → `ask(askDraft)`	`POST /api/ask`	`{sessionId, question, device}`	`{answer, manualHits, moments}`	Answer streams in word by word with source chips; opens the manual at a cited page
Suggested question chips (Ask panel, per device)	`ask(text)`	`POST /api/ask`	Same; `question` = chip text	Same	Same
Ask by voice mic (next to Ask input)	`askVoice` → `voice.toggle(Ask)` → `ask(text)`	`POST /api/transcribe` if needed; then `POST /api/ask`	WAV; then same as above	`{text}`; then same	Input shows “Listening…” and live text
Manual source chip (under an answer, e.g. “p.142 · Alarms”)	`openSource(source)` → `openManual(target)`	If not yet downloaded: `GET /manuals/{id}.pdf`	—	PDF file (cached after first download)	Manual scrolls to the page and highlights the `box` area
Manual book icon (control bar)	`toggleManual`	On first show: `GET /manuals/{id}.pdf`	—	PDF file	Shows/hides the manual panel
Device name chip (top of Capture panel)	`selectDevice(device)`	No new request (uses cached device list); the manual panel downloads the new PDF	—	—	Switches device, clears chat, swaps manual and suggestions

2.2 Buttons that only change local state

Button	Action	Effect
Ask Tundra ✦ (control bar)	`toggleAsk`	Shows/hides the Ask panel; in narrow windows switches to the Ask page
Close × (Ask / manual panel)	`toggleAsk` / `toggleManual`	Closes the panel
Zoom in / Zoom out / double tap (manual panel)	Panel-internal state	Zooms the PDF, no network
Typing in the input	`draft(text)`	Updates `askDraft`
Switch to Home Space / Full Space	XR `session.scene`	Changes space mode, no network

2.3 Automatic requests (no button)

When	API	Purpose
App start	`GET /api/devices`	Loads devices and manuals; after 1.2 s the first device becomes the “identified” device
Every 3 s	`GET /api/captures?sessionId=…`	Heartbeat (is the laptop online) + refreshes each capture’s analysis status

3. Brain API reference

The brain has 12 endpoints: the headset uses 6, the laptop dashboard uses the other 6. JSON responses carry `access-control-allow-origin: *`. Errors always return `{ "error": "…" }` with status 400, 404, 500 or 501.

3.1 GET /api/devices — device and manual catalog

Input: none.

Output: an array, one entry per device, read from `server/manuals/catalog.json`. Devices whose PDF is missing are left out.

```
[{
  "id": "pb980", "name": "Puritan Bennett 980", "manufacturer": "Medtronic",
  "category": "Ventilator", "assetTag": "ICU-VNT-0412", "serial":
"35B1800412",
  "location": "ICU 3 · Bed 7", "lastPm": "14 Jul 2026", "openWorkOrders": 1,
  "manual": { "id": "pb980-om", "title": "PB980 Operator's Manual",
"docNumber": "PT00169219",
              "kind": "Operator's manual", "file": "pb980-operators-
manual.pdf",
              "pages": 412, "url": "/manuals/pb980-om.pdf" }
}]
```

Field	Meaning
`id`	The device's unique id in the brain; `/api/ask` uses it to find the manual
`assetTag` / `serial` / `location`	Hospital asset tag, serial number, location (demo values for now; will come from the CMMS)
`lastPm` / `openWorkOrders`	Date of the last preventive maintenance, number of open work orders
`manual.docNumber`	OEM document number; shown in the app as the manual revision
`manual.pages`	Total PDF pages
`manual.url`	PDF path; the app prepends `baseUrl`

3.2 GET /manuals/{manualId}.pdf — manual file

Input: the manual id in the path; a `Range` header is supported. **Output:** an `application/pdf` file. The app downloads it once, caches it, and renders it with `PdfRenderer`.

3.3 POST /api/captures — upload one capture

Input: metadata travels in a header, and the body is the raw media bytes (empty for a voice note). Large videos stream straight up with no multipart encoding.

Header	Value
`content-type`	`image/jpeg` (photo) or `video/mp4` (video); omitted for notes

Header	Value
`x-tundra-meta`	URI-encoded JSON, below

```
{
  "sessionId": "s_1760700000000",
  "kind": "photo",
  "at": "02:14",
  "durationSec": 0,
  "text": "voice notes only: the transcript",
  "device": { "id": "pb980", "name": "…", "manufacturer": "…", "category":
"…",
              "assetTag": "…", "serial": "…", "location": "…" }
}
```

Meta field	Meaning
`sessionId`	Session id created at app start (`s_` + epoch ms); groups captures from one job
`kind`	`photo` / `video` / `note`
`at`	Time since the session started, `mm:ss`
`durationSec`	Video length in seconds; 0 for other kinds
`text`	Voice-note transcript
`device`	The current device's identity, stored as sent

Output: `201` with the full Capture record. The app reads only `id`, `tl` and `analysis.findings`.

```
{
  "id": "a1b2c3d4", "sessionId": "…", "kind": "video", "at": "02:14",
  "createdAt": "2026-10-17T09:30:00.000Z", "device": { "…": "…" },
  "file": "a1b2c3d4.mp4", "mime": "video/mp4", "bytes": 18300000,
"durationSec": 12,
  "tl": { "upload": "pending", "analysis": "pending", "index": "pending" },
  "qa": []
}
```

Capture field	Meaning
`id`	8-character id from the brain; the app stores it as `remoteId` and matches on it when polling
`file` / `mime` / `bytes`	File name in `server/data/media/`, type, size
`tl.upload`	Progress of the video upload to TwelveLabs
`tl.analysis`	Progress of AI analysis (photos and videos)
`tl.index`	Progress of video indexing (for later search)
`tl.error`	Failure reason, up to 400 characters
`analysis`	The structured maintenance record once analysis finishes; see section 5
`qa`	Questions asked about this video on the dashboard

Each `tl` step is one of: `pending` (queued), `running`, `done`, `failed`, `skipped` (not done: no TwelveLabs key, or a capture with no media such as a note).

3.4 GET /api/captures?sessionId=… — status polling

Input: query parameter `sessionId` (omit it to get everything). **Output:** that session's Captures, same shape as 3.3. The app calls it every 3 s: success means the laptop is online, and each capture's status is updated by `id` (mapping in section 4).

3.5 POST /api/ask — ask a question

Input:

```
{ "sessionId": "s_1760700000000", "question": "How do I run SST?",
  "device": { "id": "pb980", "…": "…" } }
```

Output:

```
{
  "answer": "PB980 Operator's Manual, p. 142 · Short Self Test:\n“…”\n\nAlso
see p. 143, p. 150.",
  "manualHits": [
    { "manualId": "pb980-om", "title": "PB980 Operator's Manual", "page":
142,
      "section": "Short Self Test", "text": "…", "box": [0.08, 0.31, 0.92,
0.47] }
  ],
  "moments": [ { "captureId": "a1b2c3d4", "start": 12.5, "end": 18.0, "rank":
1, "thumbnailUrl": "…" } ]
}
```

Field	Meaning
`answer`	The answer shown to the engineer: a quote of the best-matching manual passage (about 360 characters max), then other matching pages
`manualHits[].page`	Page number in the PDF, starting at 1
`manualHits[].section`	Heading of the section the passage sits in
`manualHits[].text`	Excerpt of the passage
`manualHits[].box`	Where the passage sits on the page, `[left, top, right, bottom]` as 0–1 page fractions from the top-left; the app draws the highlight from it
`moments[]`	Clips from videos of the same device that match the question; `start` / `end` in seconds, viewed on the laptop dashboard

The app turns each `manualHit` into a tappable “p.142 · Short Self Test” chip and each `moment` into a “00:12–00:18 · on laptop” chip. The question and answer are also stored in the session as a `kind:"question"` Capture.

3.6 POST /api/transcribe — speech to text

Input: a 16 kHz, mono, 16-bit WAV body with `content-type: audio/wav`. The headset records up to 20 s and stops after 1.6 s of silence. **Output:** `{ "text": "recognized words" }`. Returns `501` when no whisper model is installed.

3.7 Dashboard-only endpoints

Method and path	Input	Output
`GET /`	—	The dashboard page
`GET /api/state`	—	`{ twelveLabs, indexId, headsetSeenAt, addresses, captures[] }`: whether TwelveLabs is on, index id, last headset request time, laptop IPs, all captures
`GET /api/events`	—	SSE stream: `status` (every 5 s) and `capture` (whenever a capture is added or changes; the whole Capture)
`GET /media/{file}`	`Range` supported	Photo or video file
`POST /api/search`	`{ query }`	`{ moments[], photos[] }`, each with `match` (does it really match) and `description` (one sentence); 400 without a key
`POST /api/captures/{id}/ask`	`{ question }`	The updated Capture, with the new Q&A appended to `qa`

4. Status and error handling

Each capture on the headset shows one of five states, set by the brain's `tl.analysis` (`LocalBrain.toStatus()`).

Brain returns	Headset `SyncState`	Small text on the timeline
Request still in flight	`Sending`	—
`tl.analysis` = `skipped`	`InBrain` (stored, no further changes)	—
`tl.analysis` = `pending` / `running`	`Analyzing`	"Uploading" while `tl.upload` = `running`
`tl.analysis` = `done`	`Analyzed`	"N findings" (count of `analysis.findings`) or "No findings"

Brain returns	Headset `SyncState`	Small text on the timeline
`tl.analysis` = `failed`	`Failed`	“Analysis failed”
Upload request itself failed	`Failed`	“Not sent · laptop offline”

Online status

- `brainOnline = null`: no poll has run yet since start.
- `brainOnline = true`: the last `GET /api/captures` succeeded.
- `brainOnline = false`: a poll failed, or an upload failed.

What the app does when a request fails

Failed request	App behavior
`GET /api/devices`	Uses DemoBrain's sample device (Evita V500) and built-in manual pages
`POST /api/captures`	Marks that capture Failed; no automatic retry
`POST /api/ask`	Uses DemoBrain's scripted answer
`POST /api/transcribe`	Shows a voice error for 4 s; no note or question is created
`GET /manuals/{id}.pdf`	Manual panel stays on “Opening manual…”

After the system speech recognizer fails hard once, all voice input for the rest of that run goes straight to `/api/transcribe`.

5. Inside the brain

The brain saves each capture and replies at once; AI processing runs in the background. Results are written back to the same Capture, which the headset sees by polling and the dashboard sees over SSE. Without `TWELVELABS_API_KEY` it stores but does not analyze; add the key and restart, and earlier photos and videos are analyzed automatically.

5.1 Storage

- Records: `server/data/db.json` (writes batched, at most one per 200 ms).
- Media: `server/data/media/{id}.jpg|mp4`.

- Manual index: `server/data/manual-index/`, rebuilt when a PDF changes.

5.2 Video pipeline (TwelveLabs)

1. Upload: `assets.create` sends the MP4 to TwelveLabs and waits until ready (`tl.upload`).
2. Analyze: the Pegasus 1.6 model writes a maintenance record in a fixed JSON schema (`tl.analysis`).
3. Index: the video joins the `tundra-clinical-engineering` index (Marengo 3.0, visual + audio), so later questions and searches can find clips in it (`tl.index`, waits up to 15 min).

Photos get step 2 only (no upload, no index). Voice notes get no AI processing, but they are sent to the analysis model as context for the same session.

5.3 Fields in `analysis`

Field	Video	Photo	Meaning
`summary`	yes	yes	2–3 sentence summary (1 sentence for photos)
`findings[]`	yes	yes	Each finding: `title`, `severity` (info / minor / major / critical), `detail`; videos also have `start_time` / `end_time` in seconds. The headset shows only the count
`safety_concerns[]`	yes	yes	Issues affecting patient or staff safety
`work_type`	yes	—	Preventive maintenance, corrective repair, inspection, calibration, incoming inspection, troubleshooting, other
`device_identification`	yes	—	Model, serial, asset tag and calibration stickers visible in the footage
`steps_performed[]`	yes	—	What the technician did, with start times
`parts_and_consumables[]` / `tools_and_test_equipment[]`	yes	—	Parts, consumables, tools and test equipment seen or named

Field	Video	Photo	Meaning
`recommended_follow_up[]`	yes	—	Suggested next actions
`device_status_after`	yes	—	in_service / out_of_service / needs_follow_up / unknown
`documentation_gaps[]`	yes	—	Records an auditor would expect that the footage does not show (e.g. no electrical safety test result)
`visible_labels[]`	—	yes	`kind` and `text` of every readable label in the photo
`tags[]`	—	yes	3–6 search keywords

5.4 Manual search (answers /api/ask)

`pdfjs-dist` splits each PDF into passages with page, section and position box. A question takes the 3 best-scoring passages from the current device's manual. With TwelveLabs on, it also searches 6 video clips and keeps the top 3 from the same device. Today the answer is a quote from the manual, not text generated by a language model.

5.5 Speech to text

Runs the local `whisper-cli` with model `server/models/ggml-base.en.bin`, English only. At startup it transcribes 1 s of silence to warm up, so the first question doesn't wait about 15 s.